

Networking Part II — Exam Notes

Based on Course Material by Abdelkader BELAHCENE (April 2026)

1. How Switch Works (Layer 2)

Key Concepts

- **Switch operates at Layer 2** (Data Link) — only cares about MAC addresses, NOT IP addresses
- **MAC Address Table (CAM Table)**: Maps MAC addresses to switch ports
- **Encapsulation**: IP is inside the packet; MAC is in the frame (the “envelope”)

Practical Scenario: PC1 → PC2 (Same Subnet)

Phase I: ARP Request

```
PC1 checks: Is PC2 on same network? (using subnet mask AND operation)
→ Yes, same subnet

PC1 checks ARP cache for PC2's MAC
→ If not found: Broadcast ARP Request
  Frame:  Src MAC: PC1_MAC    Dst MAC: FF:FF:FF:FF:FF:FF (Broadcast)
  ARP:    "Who has 192.168.1.5? Tell 192.168.1.2"

Switch receives frame with Broadcast MAC
→ Floods frame out ALL ports except incoming port
```

Phase II: ARP Reply

```
PC2 recognizes its IP, responds with Unicast ARP Reply
  Frame:  Src MAC: PC2_MAC    Dst MAC: PC1_MAC

Switch learns: "PC2_MAC is on Port 2" → Records in CAM Table
PC1 learns: "192.168.1.5 → PC2_MAC" → Records in ARP Table
```

Phase III: Direct Communication

```
PC1 sends data frame:
  Frame:  Src MAC: PC1_MAC    Dst MAC: PC2_MAC
  Packet: Src IP: 192.168.1.2  Dst IP: 192.168.1.5

Switch looks up CAM Table: PC2_MAC → Port 2
→ Forwards frame ONLY to Port 2 (Unicast forwarding)
```

Practical Scenario: PC1 → PC3 (Different Subnet)

```
PC1 checks: Is PC3 on same network?
→ No, different subnet

PC1 sends to Default Gateway (Router):
  Frame:  Src MAC: PC1_MAC    Dst MAC: Gateway_Eth0_MAC
```

```
Packet: Src IP: 192.168.10.10   Dst IP: 192.168.20.10
```

Switch forwards to Gateway port

Gateway routes packet:

- Strips original frame, reads IP header
- Forwards out Eth1 to 192.168.20.0/24 network
- New Frame: Src MAC: Gateway_Eth1_MAC, Dst MAC: PC3_MAC
- Same IP header (Src: 192.168.10.10, Dst: 192.168.20.10)

Second switch forwards to PC3

Important Notes

- **Switch stays “invisible”:** Never looks at IP, only MAC
- **Layer 3 Switch:** Hybrid device (switch + router), maintains ARP table
- **Destination Host Unreachable:** Generated by OS if ARP fails, NOT by switch

2. OpenSSL & SSH

2.1 RSA Key Generation & Operations

```
# Generate RSA private key (1024, 2048, or 4096 bits)
openssl genrsa -out mykey.key 1024

# Extract public key from private key
openssl rsa -in mykey.key -pubout -out mykey.pub

# Display key details
openssl rsa -in mykey.key -text -noout
openssl rsa -pubin -in mykey.pub -text -noout

# Encrypt file with public key
openssl pkeyutl -encrypt -pubin -inkey mykey.pub -in message.txt -out message.enc

# Decrypt with private key
openssl pkeyutl -decrypt -inkey mykey.key -in message.enc -out message.dec

# Sign file with private key (signs the hash, not the file for efficiency)
openssl dgst -sha256 -sign mykey.key -out message.sig message.txt

# Verify signature with public key
openssl dgst -sha256 -verify mykey.pub -signature message.sig message.txt

# Expected output: Verified OK
```

2.2 SSH Server Configuration

```
# Check installation
sudo systemctl status ssh
```

```

# Install if missing (Debian/Ubuntu)
sudo apt install openssh-server

# Start/enable service
sudo systemctl start ssh
sudo systemctl enable ssh

# Access server
ssh login@Server_IP

# Run graphical applications (X11 forwarding)
ssh -X login@Server

# Requirements:
#   - Client gives permission (-X option)
#   - Server gives permission: X11Forwarding yes in /etc/ssh/sshd_config

# Remote copy
scp source_file login@Server:/path/to/dest
scp -r source_dir login@Server:/path/to/dest      # recursive (folders)
scp -P 4567 -r myDir login@Server:/home/user      # non-standard port (UPPERCASE P)

# Execute remote command without login
ssh login@Server -p 2345 "ls /"

# Key-based authentication (passwordless)
ssh-keygen -t rsa                                # Generate key pair
ssh-copy-id -i ~/.ssh/id_rsa.pub login@Server     # Copy public key to server
# Now login without password

```

2.3 SSH Config File: /etc/ssh/sshd_config

```

# Important directives
Port 22                # Change to custom port (e.g., 4567)
X11Forwarding yes/no   # Enable/disable graphical apps
PermitRootLogin yes/no # Security: usually set to no
PasswordAuthentication yes/no # Disable password, use keys only

```

2.4 Crontab for Automation

```

# Edit crontab file
nano mycron.txt

# Format: minute hour day month dayWeek command
# Example: Archive C programs every day at 4:20 AM
20 4 * * * tar czf mesProg.tgz *.c

# Activate
crontab mycron.txt

# List jobs
crontab -l

```

```
# Remove all jobs
crontab -r
```

3. FTP Server (vsftpd)

3.1 Installation & Service Management

```
# Install
sudo apt-get install vsftpd

# Service control
sudo systemctl status vsftpd
sudo systemctl start vsftpd
sudo systemctl stop vsftpd
sudo systemctl restart vsftpd    # After config changes
sudo systemctl enable vsftpd     # Start on boot
```

3.2 Configuration File: /etc/vsftpd.conf

```
# --- Anonymous Mode ---
anonymous_enable=YES          # Enable anonymous login
anon_upload_enable=YES        # Allow anonymous uploads (security risk!)
anon_mkdir_write_enable=YES    # Allow anonymous to create dirs

# --- Local User Mode ---
local_enable=YES              # Accept local user connections
write_enable=YES              # Allow write operations

# --- Chroot Jail (confine user to home directory) ---
chroot_local_user=YES
allow_writeable_chroot=YES     # Required if chroot dir is writable

# --- Change public folder for anonymous ---
# Modify in /etc/passwd: ftp user's home directory
# Or use: anon_root=/ftpSite

# --- Change public folder for local users ---
local_root=/ftpSite           # All local users start here
```

3.3 FTP Commands (Client Side)

```
# Connect
ftp 192.168.10.1

# Login with username/password

# Basic commands
ls          # List remote files
pwd         # Show remote directory
```

```
lpwd          # Show local directory
cd dir        # Change remote directory
lcd dir       # Change local directory
put file.txt  # Upload file
get file.txt  # Download file
mget *.txt    # Download multiple files
mput *.txt    # Upload multiple files
```

3.4 Important Notes

- **Port used:** 21
 - **Anonymous user:** Defined in `/etc/passwd` (usually `ftp` user)
 - **Local users:** Access their home directory by default
 - **Jail:** User cannot navigate outside their assigned directory
-

4. DNS Server (Bind9)

4.1 Hostname & Domain Setup

```
# Check current hostname/domain
hostnamectl

# Set hostname with domain
sudo hostnamectl set-hostname server.bela.lan

# Verify
hostnamectl
```

4.2 Static IP Configuration

```
# Using nmcli (NetworkManager)
nmcli connection show
nmcli connection modify "ConnectionName" ipv4.addresses 192.168.1.100/24
nmcli connection modify "ConnectionName" ipv4.gateway 192.168.1.1
nmcli connection modify "ConnectionName" ipv4.dns "192.168.1.100,8.8.8.8"
nmcli connection modify "ConnectionName" ipv4.method manual
nmcli connection down "ConnectionName"
nmcli connection up "ConnectionName"

# /etc/resolv.conf (auto-generated by NetworkManager)
# To prevent overwriting:
sudo chattr +i /etc/resolv.conf

# Content of /etc/resolv.conf:
search bela.lan
nameserver 192.168.1.100 # Local DNS
nameserver 8.8.8.8       # External DNS (Google)
```

4.3 Install & Configure Bind9

```
# Install
sudo apt-get install bind9 bind9utils

# Stop firewall (for testing)
sudo systemctl disable ufw
sudo systemctl stop ufw

# Configuration files location: /etc/bind
```

4.4 Zone Configuration: /etc/bind/named.conf.local

```
zone "bela.lan" {
    type master;
    file "/etc/bind/db.bela.lan";
};
```

4.5 Zone File: /etc/bind/db.bela.lan

```
$TTL 604800
@ IN SOA belaLap.bela.lan. admin.bela.lan. (
    3          ; Serial (increment on each change)
    604800     ; Refresh
    86400      ; Retry
    2419200    ; Expire
    604800 )   ; Negative Cache TTL
;
@ IN NS  belaLap.bela.lan.
@ IN A   192.168.10.10
belaLap IN A   192.168.10.10
oldLap  IN A   192.168.10.20
myWeb   IN A   192.168.10.30
```

4.6 Validation & Restart

```
# Check configuration syntax
sudo named-checkconf
sudo named-checkzone bela.lan /etc/bind/db.bela.lan

# Restart service
sudo systemctl restart bind9
```

4.7 Multi-LAN DNS Configuration

```
# Edit /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";
    recursion yes;
    allow-query {
        192.168.10.0/24;
```

```
        192.168.20.0/24;
        192.168.30.0/24;
    };
    allow-recursion {
        192.168.10.0/24;
        192.168.20.0/24;
        192.168.30.0/24;
    };
};
```

5. DHCP Server (isc-dhcp-server)

5.1 Installation & Interface Setup

```
# Install
sudo apt-get install isc-dhcp-server

# Specify interface in /etc/default/isc-dhcp-server
INTERFACESv4="eth0"
```

5.2 Main Config: /etc/dhcp/dhcpd.conf

```
# Global settings
default-lease-time 600;      # 10 minutes (test), 86400 for production (24h)
max-lease-time 7200;        # 2 hours maximum
authoritative;               # Official DHCP for this network

# Subnet declaration
subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.5 192.168.10.50;      # Dynamic IP pool
    option subnet-mask 255.255.255.0;
    option broadcast-address 192.168.10.255;
    option domain-name "bela.lan";
    option routers 192.168.10.10;          # Gateway
    option domain-name-servers 192.168.10.10, 8.8.8.8; # DNS servers
}

# Static reservations (servers)
host webserver {
    hardware ethernet B8:8D:12:01:4B:A8;
    fixed-address 192.168.10.70;
}

host fileserver {
    hardware ethernet 20:89:84:1F:51:BE;
    fixed-address 192.168.10.80;
}
```

5.3 Ports

- Client → Server: UDP 68 → UDP 67
 - Server → Client: UDP 67 → UDP 68
-

6. Web Server (Apache2)

6.1 Installation & Structure

```
# Install
sudo apt install apache2

# Start/Check
sudo systemctl start apache2
sudo systemctl status apache2
sudo systemctl enable apache2

# Important environment variables
# See /etc/apache2/envvars
# APACHE_LOG_DIR=/var/log/apache2
```

6.2 Directory Structure

```
/etc/apache2/
├─ apache2.conf          # Main config file
├─ conf-available/
├─ conf-enabled/
├─ envvars
├─ magic
├─ mods-available/      # 144+ available modules
├─ mods-enabled/        # 34+ enabled by default
├─ ports.conf           # Listening ports
├─ sites-available/     # Site configs
└─ sites-enabled/       # Symlinks to enabled sites
```

6.3 Main Config: /etc/apache2/apache2.conf

```
# Key directives
ServerAdmin bela@localhost
DocumentRoot /var/www/html

# Directory permissions
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>

# Options explained:
# Indexes: Show directory listing if no index file
```



```
# FollowSymLinks: Follow symbolic links
# AllowOverride None: Ignore .htaccess files
# AllowOverride All: Process .htaccess files

# Logs
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
```

6.4 Ports Config: `/etc/apache2/ports.conf`

```
Listen 80
<IfModule ssl_module>
    Listen 443
</IfModule>

# Additional ports can be added:
Listen 8000
Listen 8443
```

6.5 Virtual Hosts

```
# Create new site config
sudo nano /etc/apache2/sites-available/html2.conf
```

```
<VirtualHost *:8000>
    ServerAdmin bela@localhost
    DocumentRoot /var/www/html2
    ErrorLog ${APACHE_LOG_DIR}/error.log
</VirtualHost>
```

```
# Enable site
sudo a2ensite html2.conf
# Add Listen 8000 to ports.conf if not present
sudo systemctl restart apache2
# Access: http://localhost:8000
```

6.6 User Directories (`public_html`)

```
# Enable module
sudo a2enmod userdir
sudo systemctl restart apache2
```

Config file: `/etc/apache2/mods-available/userdir.conf`

```
UserDir public_html
UserDir disabled root

<Directory /home/*/public_html>
    AllowOverride FileInfo AuthConfig Limit Indexes
    Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExec
    Require method GET POST OPTIONS
```

```
</Directory>
```

User setup:

```
# As user (e.g., user22):
mkdir ~/public_html
chmod 711 ~           # Home must be traversable
chmod 755 ~/public_html # Public dir must be readable

# Create page
echo "<h1>My Page</h1>" > ~/public_html/index.html

# Access: http://server/~user22/
```

6.7 Directory Protection (.htaccess)

```
# Create password file
htpasswd -c /home/user/.htpasswd username

# .htaccess file in protected directory:
AuthType Basic
AuthName "Password Required"
AuthUserFile /home/user/.htpasswd
Require valid-user
```

6.8 Security Configurations

```
# Hide version and OS
# Edit /etc/apache2/conf-enabled/security.conf
ServerSignature Off
ServerTokens Prod

# Disable directory listing
<Directory /var/www/html>
    Options -Indexes
</Directory>

# For user directories
<Directory "/home/*/public_html">
    Options -Indexes
</Directory>

# Enable PHP for user directories
# Comment last lines in /etc/apache2/mods-enabled/phpX.conf
```

7. Firewall (iptables)

7.1 Tables & Chains

Tables:

- filter: Default, packet filtering (ACCEPT/DROP)
- nat: Address translation (DNAT, SNAT)
- mangle: Packet modification (QoS, TTL, TOS)

Chains:

- INPUT: Packets destined for local machine
- OUTPUT: Packets leaving local machine
- FORWARD: Packets passing through (router/gateway)
- PREROUTING: Before routing decision
- POSTROUTING: After routing decision

7.2 Basic Commands

```
# List rules
sudo iptables -L           # All chains
sudo iptables -L INPUT    # Specific chain
sudo iptables -S           # List by specification
sudo iptables -t nat -L    # NAT table

# Flush rules (remove all except defaults)
sudo iptables -F

sudo iptables -X           # Delete custom chains

# Set default policy
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT DROP

# Save rules
sudo /sbin/iptables-save

# OR install persistent:
sudo apt-get install iptables-persistent
sudo netfilter-persistent save
```

7.3 Rule Syntax

```
# Append rule
iptables -A CHAIN [conditions] -j TARGET

# Insert at beginning
iptables -I CHAIN [conditions] -j TARGET

# Delete rule
```

```
iptables -D CHAIN rule_number

# Common conditions:
-p tcp/udp/icmp           # Protocol
-s 192.168.10.0/24        # Source
-d 192.168.10.10          # Destination
--dport 80                # Destination port
--sport 80                 # Source port
-i eth0                   # Input interface
-o eth0                   # Output interface
-m state --state ESTABLISHED,RELATED # Connection state
```

7.4 Complete Firewall Setup Example

```
#!/bin/bash

# Initialize: Flush everything
iptables -t filter -F
iptables -t filter -X
iptables -t nat -F
iptables -t nat -X

# Default policy: DENY ALL
iptables -t filter -P INPUT DROP
iptables -t filter -P FORWARD DROP
iptables -t filter -P OUTPUT DROP

# 1. Allow localhost (loopback)
iptables -t filter -A INPUT -i lo -j ACCEPT
iptables -t filter -A OUTPUT -o lo -j ACCEPT

# 2. Allow established/related connections
iptables -t filter -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -t filter -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# 3. Allow HTTP (port 80) from anywhere
iptables -t filter -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -t filter -A OUTPUT -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT

# 4. Allow HTTPS (port 443)
iptables -t filter -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -t filter -A OUTPUT -p tcp --sport 443 -m state --state ESTABLISHED -j ACCEPT

# 5. Allow SSH (port 22) from anywhere
iptables -t filter -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -t filter -A OUTPUT -p tcp --sport 22 -m state --state ESTABLISHED -j ACCEPT

# 6. Allow SSH outgoing (connect to remote servers)
iptables -t filter -A OUTPUT -p tcp --dport 22 -j ACCEPT

# 7. Allow DNS (port 53)
```

```

iptables -t filter -A OUTPUT -p udp --dport 53 -j ACCEPT
iptables -t filter -A INPUT -p udp --dport 53 -j ACCEPT

# 8. Allow Ping (ICMP)
iptables -t filter -A INPUT -p icmp -j ACCEPT
iptables -t filter -A OUTPUT -p icmp -j ACCEPT

# 9. Allow DHCP
iptables -t filter -A OUTPUT -p udp --dport 68 -j ACCEPT
iptables -t filter -A INPUT -p udp --dport 68 -j ACCEPT

# 10. Allow SMTP (Mail)
iptables -t filter -A INPUT -p tcp --dport 25 -j ACCEPT
iptables -t filter -A OUTPUT -p tcp --dport 25 -j ACCEPT

# 11. Allow NTP (Time sync)
iptables -t filter -A OUTPUT -p udp --dport 123 -j ACCEPT

# 12. Forwarding rules (if acting as gateway)
# Allow PC1 (192.168.10.10) to communicate with PC3 (192.168.20.10)
iptables -t filter -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -j ACCEPT
iptables -t filter -A FORWARD -s 192.168.20.10 -d 192.168.10.10 -j ACCEPT

```

7.5 Specific Rules from Course

```

# Block specific IP
iptables -A INPUT -s 8.8.8.8 -j DROP

# Block subnet
iptables -A INPUT -s 10.10.10.0/24 -j DROP

# Block interface + IP
iptables -A INPUT -i eth0 -s 192.168.10.55 -j DROP

# Allow forwarding between interfaces
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT

# Allow web only from specific IP
iptables -A INPUT -p tcp -s 192.168.10.10 --dport 80 -j ACCEPT

```

7.6 Connection States

- **ESTABLISHED:** Packet belongs to ongoing connection
- **RELATED:** New connection linked to existing one (e.g., FTP data connection)
- **NEW:** First packet of new connection
- **INVALID:** Packet doesn't belong to any known connection

8. VLAN, Bridge & Router Configuration

8.1 VLAN Concepts

- **VLAN:** Logical broadcast domain within physical network
- **Access Port:** Untagged, belongs to single VLAN
- **Trunk Port:** Tagged, carries multiple VLANs
- **802.1Q Tag:** 4 bytes added to Ethernet frame (VLAN ID: 12 bits)

8.2 Linux Bridge (Simulating Switch)

```
# Create bridge (switch)
sudo ip link add br0 type bridge vlan_filtering 1
sudo ip link set br0 up

# Add interface to bridge
sudo ip link set eth0 master br0
```

8.3 Network Namespaces (Simulating PCs)

```
# Create namespace (virtual PC)
sudo ip netns add PC1

# Create veth pair (virtual cable)
sudo ip link add veth1 type veth peer name veth1br

# Connect one end to namespace
sudo ip link set veth1 netns PC1

# Connect other end to bridge
sudo ip link set veth1br master br0
sudo ip link set veth1br up

# Assign to VLAN (untagged)
sudo bridge vlan add dev veth1br vid 10 pvid untagged

# Configure IP in namespace
sudo ip netns exec PC1 ip addr add 192.168.10.1/24 dev veth1
sudo ip netns exec PC1 ip link set lo up
sudo ip netns exec PC1 ip link set veth1 up
```

8.4 Router Namespace

```
# Create router namespace
sudo ip netns add R1

# Create trunk link to bridge
sudo ip link add vethR type veth peer name vethRbr
sudo ip link set vethR netns R1
sudo ip link set vethRbr master br0
sudo ip link set vethRbr up
```

```

# Configure trunk VLANs
sudo bridge vlan add dev vethRbr vid 10
sudo bridge vlan add dev vethRbr vid 20

# Enable interface in router
sudo ip netns exec R1 ip link set lo up
sudo ip netns exec R1 ip link set vethR up

# Create VLAN sub-interfaces
sudo ip netns exec R1 ip link add link vethR name vethR.10 type vlan id 10
sudo ip netns exec R1 ip addr add 192.168.10.254/24 dev vethR.10
sudo ip netns exec R1 ip link set vethR.10 up

sudo ip netns exec R1 ip link add link vethR name vethR.20 type vlan id 20
sudo ip netns exec R1 ip addr add 192.168.20.254/24 dev vethR.20
sudo ip netns exec R1 ip link set vethR.20 up

# Enable IP forwarding
sudo ip netns exec R1 sysctl -w net.ipv4.ip_forward=1

# Set default gateway on PC
sudo ip netns exec PC1 ip route add default via 192.168.10.254

```

8.5 Policy Routing (Multiple NICs)

```

# Add routing tables
sudo echo "100 lan1" >> /usr/share/iproute2/routing
sudo echo "200 lan2" >> /usr/share/iproute2/routing

# Routes for LAN1
sudo ip route add 192.168.10.0/24 dev eth0 src 192.168.10.10 table lan1
sudo ip route add default via 192.168.10.254 dev eth0 table lan1

# Routes for LAN2
sudo ip route add 192.168.20.0/24 dev eth1 src 192.168.20.10 table lan2
sudo ip route add default via 192.168.20.254 dev eth1 table lan2

# Policy rules
sudo ip rule add from 192.168.10.10 table lan1
sudo ip rule add from 192.168.20.10 table lan2

# Verify
sudo ip rule show
sudo ip route show table lan1

```

9. Exam-Style Questions

Question I: How Switch Works (4 pts)

Consider a LAN with:

- PC1 → 192.168.10.10/24
- PC2 → 192.168.10.20/24
- PC3 → 192.168.20.10/24
- Gateway Eth0 → 192.168.10.1
- Gateway Eth1 → 192.168.20.1

In few words, what do the PCs and the switch do in each case? Precise protocol, address, source, destination.

a) PC1 wants to send a packet to PC2

Solution

1. PC1 checks destination IP (192.168.10.20) with subnet mask (/24) → same subnet
2. PC1 checks ARP cache for PC2's MAC
3. If not found: Broadcast ARP Request
 - Frame: Src MAC=PC1, Dst MAC=FF:FF:FF:FF:FF:FF
 - ARP: "Who has 192.168.10.20? Tell 192.168.10.10"
4. Switch floods ARP Request to all ports (broadcast)
5. PC2 responds with Unicast ARP Reply
 - Frame: Src MAC=PC2, Dst MAC=PC1
6. Switch learns PC2's MAC → Port mapping in CAM table
7. PC1 sends data frame
 - Frame: Src MAC=PC1, Dst MAC=PC2
 - Packet: Src IP=192.168.10.10, Dst IP=192.168.10.20
8. Switch forwards unicast to PC2's port only

b) PC1 wants to send a packet to PC3

Solution

1. PC1 checks destination IP (192.168.20.10) with subnet mask → different subnet
2. PC1 sends to Default Gateway (192.168.10.1)
3. PC1 ARPs for Gateway MAC if not cached
4. PC1 sends frame:
 - Frame: Src MAC=PC1, Dst MAC=Gateway_Eth0
 - Packet: Src IP=192.168.10.10, Dst IP=192.168.20.10
5. Switch forwards to Gateway port
6. Gateway receives, strips frame, reads IP header
7. Gateway routes to 192.168.20.0/24 via Eth1
8. Gateway ARPs for PC3 MAC on Eth1 side
9. Gateway sends new frame:
 - Frame: Src MAC=Gateway_Eth1, Dst MAC=PC3
 - Packet: Src IP=192.168.10.10, Dst IP=192.168.20.10 (unchanged)
10. Second switch forwards to PC3

Question II: vsFTP Configuration (5 pts)

We have an FTP server on 192.168.10.1, local machine 192.168.10.10. Account: user11.

On Server:

- Check installation, start if not started
- Precise config file name
- Accept local user login with write possibility
- Accept anonymous login
- Put local user in Jail
- Modify public folder to /ftpSite

On Local Machine:

- Try connection as user

Solution

```
# === ON SERVER (192.168.10.1) ===

# Check and start
sudo systemctl status vsftpd
sudo systemctl start vsftpd
sudo systemctl enable vsftpd

# Config file: /etc/vsftpd.conf
sudo nano /etc/vsftpd.conf

# Add/modify these lines:
anonymous_enable=YES
local_enable=YES
write_enable=YES
chroot_local_user=YES
allow_writeable_chroot=YES
local_root=/ftpSite

# Create and set permissions
sudo mkdir -p /ftpSite
sudo chmod 755 /ftpSite

# Restart
sudo systemctl restart vsftpd

# === ON LOCAL MACHINE (192.168.10.10) ===

ftp 192.168.10.1
# Enter username: user11
# Enter password: <password>
```

```
# Try commands: ls, put file.txt, get file.txt
```

Question III: Apache2 Server (6 pts)

Web server (myServ) 192.168.10.1, local machine (myLocal) 192.168.10.10. Account: user22.

On myServ:

- Start web server
- Precise main config file full path
- Public site folder: /var/ensta/html
- From myServ, browse the web
- From myLocal, browse the web
- Enable directory listing (show files)
- Allow any user on myServ to create own page

On myLocal: What should user22 do to create his own web page?

Solution

```
# === ON SERVER (myServ) ===

# Start
sudo systemctl start apache2
sudo systemctl enable apache2

# Main config file: /etc/apache2/apache2.conf

# Change document root
sudo nano /etc/apache2/sites-available/000-default.conf
# Modify: DocumentRoot /var/ensta/html

# Add directory permissions:
<Directory /var/ensta/html>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>

# Create directory
sudo mkdir -p /var/ensta/html
sudo chown -R www-data:www-data /var/ensta/html
sudo chmod -R 755 /var/ensta/html

# Enable user directories
sudo a2enmod userdir
sudo systemctl restart apache2
```

```

# Test from myServ
firefox http://localhost
# or
curl http://192.168.10.1

# Test from myLocal
firefox http://192.168.10.1

# === FOR user22 TO CREATE OWN PAGE ===

# As user22 on myServ:
mkdir -p ~/public_html
chmod 711 ~          # Home traversable
chmod 755 ~/public_html # Public readable

# Create page
echo "<h1>user22's Page</h1>" > ~/public_html/index.html

# Access from anywhere: http://192.168.10.1/~user22/

```

Question IV: Firewall (5 pts)

Configure Gateway with iptables. Web (80) and SSH (22) servers running on Gateway.

Base rule (remains enabled): Deny all traffic

Independent rules for:

1. Accept all traffic inside server (localhost)
2. Accept Web from any machine (including local)
3. Accept Web only from PC1 (192.168.10.10)
4. Accept SSH access to server
5. Allow communication with PC2 (192.168.10.20)
6. Allow communication between PC1 (192.168.10.10) and PC3 (192.168.20.10)

Solution

```

# Base: Deny all
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT DROP

# 1. Accept localhost traffic
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

# 2. Accept Web from any machine
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

```

```

sudo iptables -A OUTPUT -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT

# 3. Accept Web only from PC1
sudo iptables -A INPUT -p tcp -s 192.168.10.10 --dport 80 -j ACCEPT

# 4. Accept SSH access
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A OUTPUT -p tcp --sport 22 -m state --state ESTABLISHED -j ACCEPT

# 5. Allow communication with PC2
sudo iptables -A INPUT -s 192.168.10.20 -j ACCEPT
sudo iptables -A OUTPUT -d 192.168.10.20 -j ACCEPT

# 6. Allow communication between PC1 and PC3 (forwarding)
sudo iptables -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -j ACCEPT
sudo iptables -A FORWARD -s 192.168.20.10 -d 192.168.10.10 -j ACCEPT

# Allow established connections for all chains
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

```

Question V: DNS Server (4 pts)

Configure Bind9 DNS server for domain “ensta.lan” with:

- Server hostname: dnsServ.ensta.lan, IP: 192.168.10.10
- PC1: 192.168.10.20
- PC2: 192.168.10.30
- Web server: 192.168.10.40
- Forward unknown queries to 8.8.8.8

Solution

```

# Set hostname
sudo hostnamectl set-hostname dnsServ.ensta.lan

# Install
sudo apt-get install bind9 bind9utils

# Configure zone in /etc/bind/named.conf.local
zone "ensta.lan" {
    type master;
    file "/etc/bind/db.ensta.lan";
};

# Create zone file /etc/bind/db.ensta.lan
$TTL 604800
@ IN SOA dnsServ.ensta.lan. admin.ensta.lan. (

```

```

        1          ; Serial
        604800     ; Refresh
        86400      ; Retry
        2419200    ; Expire
        604800 )   ; Negative Cache TTL
;
@      IN  NS    dnsServ.ensta.lan.
@      IN  A     192.168.10.10
dnsServ IN  A     192.168.10.10
PC1    IN  A     192.168.10.20
PC2    IN  A     192.168.10.30
webServ IN  A     192.168.10.40

# Configure forwarding in /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";
    recursion yes;
    allow-query { 192.168.10.0/24; };
    forwarders { 8.8.8.8; };
};

# Check and restart
sudo named-checkconf
sudo named-checkzone ensta.lan /etc/bind/db.ensta.lan
sudo systemctl restart bind9

# Client /etc/resolv.conf
search ensta.lan
nameserver 192.168.10.10
nameserver 8.8.8.8

```

Question VI: DHCP Server (4 pts)

Configure DHCP for LAN 192.168.10.0/24 with:

- IP range: 192.168.10.100 to 192.168.10.150
- Gateway: 192.168.10.1
- DNS: 192.168.10.10 (local), 8.8.8.8 (external)
- Domain: ensta.lan
- Static reservation for Web server (MAC: AA:BB:CC:DD:EE:FF) → 192.168.10.50

Solution

```

# Install
sudo apt-get install isc-dhcp-server

# Set interface in /etc/default/isc-dhcp-server
INTERFACESv4="eth0"

```

```
# Main config /etc/dhcp/dhcpd.conf
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.100 192.168.10.150;
    option subnet-mask 255.255.255.0;
    option broadcast-address 192.168.10.255;
    option routers 192.168.10.1;
    option domain-name-servers 192.168.10.10, 8.8.8.8;
    option domain-name "ensta.lan";
}

host webserver {
    hardware ethernet AA:BB:CC:DD:EE:FF;
    fixed-address 192.168.10.50;
}

# Restart
sudo systemctl restart isc-dhcp-server
```

Question VII: VLAN & Routing (6 pts)

Create a virtual network with:

- Bridge br0 (VLAN switch)
- PC1 in VLAN 10, IP: 192.168.10.2/24
- PC2 in VLAN 10, IP: 192.168.10.3/24
- PC3 in VLAN 20, IP: 192.168.20.2/24
- Router R1 with VLAN interfaces: 192.168.10.1 (VLAN10), 192.168.20.1 (VLAN20)
- Enable inter-VLAN routing

Solution

```
# Create bridge
sudo ip link add br0 type bridge vlan_filtering 1
sudo ip link set br0 up

# === PC1 (VLAN 10) ===
sudo ip netns add PC1
sudo ip link add veth1 type veth peer name veth1br
sudo ip link set veth1 netns PC1
sudo ip link set veth1br master br0
sudo ip link set veth1br up
sudo bridge vlan add dev veth1br vid 10 pvid untagged
sudo ip netns exec PC1 ip addr add 192.168.10.2/24 dev veth1
sudo ip netns exec PC1 ip link set lo up
```

```

sudo ip netns exec PC1 ip link set veth1 up

# === PC2 (VLAN 10) ===
sudo ip netns add PC2
sudo ip link add veth2 type veth peer name veth2br
sudo ip link set veth2 netns PC2
sudo ip link set veth2br master br0
sudo ip link set veth2br up
sudo bridge vlan add dev veth2br vid 10 pvid untagged
sudo ip netns exec PC2 ip addr add 192.168.10.3/24 dev veth2
sudo ip netns exec PC2 ip link set lo up
sudo ip netns exec PC2 ip link set veth2 up

# === PC3 (VLAN 20) ===
sudo ip netns add PC3
sudo ip link add veth3 type veth peer name veth3br
sudo ip link set veth3 netns PC3
sudo ip link set veth3br master br0
sudo ip link set veth3br up
sudo bridge vlan add dev veth3br vid 20 pvid untagged
sudo ip netns exec PC3 ip addr add 192.168.20.2/24 dev veth3
sudo ip netns exec PC3 ip link set lo up
sudo ip netns exec PC3 ip link set veth3 up

# === Router R1 ===
sudo ip netns add R1
sudo ip link add vethR type veth peer name vethRbr
sudo ip link set vethR netns R1
sudo ip link set vethRbr master br0
sudo ip link set vethRbr up

# Trunk configuration
sudo bridge vlan add dev vethRbr vid 10
sudo bridge vlan add dev vethRbr vid 20

sudo ip netns exec R1 ip link set lo up
sudo ip netns exec R1 ip link set vethR up

# VLAN sub-interfaces
sudo ip netns exec R1 ip link add link vethR name vethR.10 type vlan id 10
sudo ip netns exec R1 ip addr add 192.168.10.1/24 dev vethR.10
sudo ip netns exec R1 ip link set vethR.10 up

sudo ip netns exec R1 ip link add link vethR name vethR.20 type vlan id 20
sudo ip netns exec R1 ip addr add 192.168.20.1/24 dev vethR.20
sudo ip netns exec R1 ip link set vethR.20 up

# Enable routing
sudo ip netns exec R1 sysctl -w net.ipv4.ip_forward=1

# Default routes on PCs

```

```
sudo ip netns exec PC1 ip route add default via 192.168.10.1
sudo ip netns exec PC2 ip route add default via 192.168.10.1
sudo ip netns exec PC3 ip route add default via 192.168.20.1

# Test
sudo ip netns exec PC1 ping 192.168.20.2
```

Quick Reference: File Locations

Service	Main Config File
SSH	/etc/ssh/sshd_config
vsFTP	/etc/vsftpd.conf
Apache2	/etc/apache2/apache2.conf
Apache2 Sites	/etc/apache2/sites-available/ 000-default.conf
DNS (Bind9)	/etc/bind/named.conf.local
DNS Zone	/etc/bind/db.<domain>
DNS Options	/etc/bind/named.conf.options
DHCP	/etc/dhcp/dhcpd.conf
DHCP Interface	/etc/default/isc-dhcp-server
Firewall	iptables (runtime) / iptables-save (persistent)
Network DNS	/etc/resolv.conf
Hostname	Set via hostnamectl

Quick Reference: Common Ports

Protocol	Port	Description
HTTP	80	Web server
HTTPS	443	Secure web
SSH	22	Secure shell
FTP	21	File transfer
DNS	53	Domain name service
DHCP	67/68	Dynamic host config
SMTP	25	Mail transfer
NTP	123	Network time

Good luck with your exam!